

Meccanismi di sincronizzazione

Alessia Smeralda Brunello

Matricola: 544964

Hands-On 11

Indice

1	Introduzione	2
2	Definizione del problema	2
3	Metodologia	3
3.1	Esercizio 1 - Assenza di sincronizzazione	3
3.2	Esercizio 2 - Mutua esclusione con mutex	3
3.3	Esercizio 3 - Controllo concorrente con semaforo	3
4	Risultati sperimentali	4
4.1	Assenza di sincronizzazione	4
4.2	Utilizzo del mutex	4
4.3	Utilizzo del semaforo	4
5	Confronto	4
6	Conclusioni	5

1 Introduzione

Nel presente laboratorio vengono analizzati i principali problemi legati alla concorrenza tra thread, con particolare attenzione ai meccanismi di sincronizzazione impiegati per garantire la correttezza nell'accesso a risorse condivise.

In ambienti concorrenti, più thread possono accedere simultaneamente a una stessa risorsa, rendendo necessario l'utilizzo di opportuni strumenti per evitare inconsistenze nei dati.

In particolare, si analizza il fenomeno delle *race condition*, ovvero situazioni in cui il risultato di un'esecuzione dipende dall'ordine temporale non deterministico delle operazioni.

A tal fine, sono stati sviluppati tre programmi distinti:

- un programma privo di meccanismi di sincronizzazione;
- un programma che utilizza un mutex per garantire la mutua esclusione;
- un programma che utilizza un semaforo per limitare l'accesso concorrente.

L'obiettivo è osservare e confrontare il comportamento dei tre approcci, evidenziando il compromesso tra correttezza e parallelismo.

2 Definizione del problema

Il problema considerato consiste nella gestione concorrente di una variabile condivisa (**counter**) da parte di più thread.

Ogni thread esegue un numero fissato di incrementi, pari a:

$$MAX_VALUE = 10000$$

Il valore finale atteso della variabile è quindi:

$$counter = NUM_THREADS \times MAX_VALUE$$

L'operazione di incremento non è atomica, in quanto composta da tre operazioni distinte:

```
temp = counter;  
temp++;  
counter = temp;
```

Questa sequenza può essere interrotta da altri thread, causando interferenze e perdita di aggiornamenti.

3 Metodologia

3.1 Esercizio 1 - Assenza di sincronizzazione

Nel primo programma non viene utilizzato alcun meccanismo di sincronizzazione. 10 thread accedono simultaneamente alla variabile condivisa.

Ogni thread esegue le seguenti operazioni:

- lettura del valore corrente della variabile;
- incremento del valore;
- scrittura del nuovo valore.

L'assenza di coordinamento tra i thread introduce una condizione di competizione (*race condition*), poiché più thread possono accedere contemporaneamente alla stessa risorsa.

3.2 Esercizio 2 - Mutua esclusione con mutex

Nel secondo programma viene introdotto un mutex per proteggere la sezione critica.

L'accesso alla variabile condivisa è racchiuso tra le seguenti operazioni:

```
pthread_mutex_lock(&mutex);  
...  
pthread_mutex_unlock(&mutex);
```

Questo meccanismo garantisce la mutua esclusione, consentendo a un solo thread alla volta di accedere alla sezione critica ed eliminando le condizioni di competizione.

3.3 Esercizio 3 - Controllo concorrente con semaforo

Nel terzo programma viene utilizzato un semaforo per limitare il numero di thread che possono accedere contemporaneamente alla sezione critica.

Il semaforo è inizializzato come segue:

```
sem_init(&access_limiter, 0, 2);
```

Ogni thread richiede l'accesso tramite:

```
sem_wait(&access_limiter);
```

e lo rilascia con:

```
sem_post(&access_limiter);
```

Questo approccio consente un accesso concorrente controllato, limitando il numero di thread attivi nella sezione critica.

4 Risultati sperimentali

4.1 Assenza di sincronizzazione

Il comportamento osservato è non deterministico. Dall'analisi dell'output si rileva che:

- più thread leggono lo stesso valore della variabile condivisa;
- il valore finale risulta inferiore a quello atteso;
- alcuni incrementi vengono persi.

Questi effetti sono riconducibili alla presenza di race condition.

4.2 Utilizzo del mutex

L'introduzione del mutex produce un comportamento corretto e deterministico. In particolare:

- il valore finale della variabile corrisponde a quello atteso;
- non si verificano sovrascritture;
- gli accessi alla risorsa sono serializzati.

Il mutex garantisce quindi la correttezza dell'esecuzione.

4.3 Utilizzo del semaforo

L'utilizzo del semaforo evidenzia un comportamento intermedio. Si osserva che:

- un massimo di due thread accede contemporaneamente alla sezione critica;
- non si verificano errori evidenti nell'esecuzione;
- il livello di parallelismo è maggiore rispetto al caso del mutex.

Tuttavia, la presenza di più thread nella sezione critica implica che le race condition non siano completamente eliminate.

5 Confronto

Approccio	Sicurezza	Parallelismo	Complessità
No Sync	Bassa	Alta	Bassa
Mutex	Alta	Bassa	Media
Semaforo	Media	Media	Alta

I risultati evidenziano come la scelta del meccanismo di sincronizzazione rappresenti un compromesso tra correttezza e prestazioni.

6 Conclusioni

Il laboratorio ha consentito di analizzare sperimentalmente i problemi legati alla concorrenza tra thread e l'importanza dei meccanismi di sincronizzazione.

L'assenza di sincronizzazione ha evidenziato chiaramente il problema delle race condition, mostrando comportamenti non deterministici e risultati errati.

L'introduzione del mutex ha garantito la mutua esclusione, assicurando la correttezza dell'esecuzione al costo di una riduzione del parallelismo.

L'utilizzo del semaforo ha invece mostrato un approccio intermedio, in grado di bilanciare parallelismo e controllo dell'accesso, pur senza garantire completamente la sicurezza delle operazioni.

In conclusione, la gestione efficace della concorrenza richiede una scelta consapevole dei meccanismi di sincronizzazione, in funzione degli obiettivi del sistema, quali correttezza, efficienza e scalabilità.